SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I Ayesha thameema banu (192524321) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Ayesha Thameema Banu

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**

   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Graph:

A → B, C

B → D, E

C → F, G

Table: BFS Dry Run

Iteration	Queue After Expansion	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

1. BFS Traversal Order:

A → B → C → D → E → F → G

2. Queue Contents After Each Iteration:

The queue follows the FIFO (First In, First Out) principle. The first inserted node is expanded first. The queue contents after every iteration are shown in the table above.

2. Depth-First Search (DFS)

Table: DFS Dry Run

Iteration	Stack After Expansion	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C
6	G	F
7	Empty	G

DFS Traversal Order:

A → B → D → E → C → F → G

Explanation:

Depth-First Search explores one branch as deeply as possible before moving to another branch. It uses a stack to store nodes that need to be visited. In this graph, DFS first explores the left branch and then moves to the right branch.

3. Eight-Puzzle Problem Using BFS

Initial State

```
1 3 6
5 0 2
4 7 8
```

Goal State

```
1 2 3
4 5 6
7 8 0
```

BFS Solution Path

Iteration 1 – Initial State

```
1 3 6
5 0 2
4 7 8
```

Iteration 2

```
1 3 6
5 2 0
4 7 8
```

Iteration 3

```
1 3 0
5 2 6
4 7 8
```

Iteration 4

```
1 0 3
5 2 6
4 7 8
```

Iteration 5

```
1 2 3
5 0 6
4 7 8
```

Iteration 6

```
1 2 3
0 5 6
4 7 8
```

Iteration 7

```
1 2 3
```

```
4 5 6
0 7 8
```

Iteration 8

```
1 2 3
4 5 6
7 0 8
```

Iteration 9 – Goal State

```
1 2 3
4 5 6
7 8 0
```

Answers to the Questions

1. Which node is expanded in each iteration?

The current state is expanded at each iteration. BFS explores states level by level, starting from the initial state. The sequence above shows a shortest path to the goal, requiring eight moves.

2. How many states are generated in total?

There are 9 states in the solution path, including the initial state and the goal state. The total number of states explored by BFS may be larger because BFS also examines other possible states before reaching the goal.

3. Write the complete solution path from the initial state to the goal state.

The solution path is:

Initial State → State 2 → State 3 → State 4 → State 5 → State 6 → State 7 → State 8 → Goal State.

The blank tile moves right, up, up, left, down, down, right and right to reach the goal state.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS explores all states at one depth before moving to the next depth. Since each move has the same cost, the first goal state reached has the minimum number of moves. Therefore, BFS guarantees a shortest solution when a solution exists.

Conclusion

Breadth-First Search explores nodes level by level using a queue, while Depth-First Search explores one branch deeply using a stack. In the eight-puzzle problem, BFS explores states systematically and finds a shortest path to the goal because every move has equal cost.